

SCGSR Mid-award Progress Report Additional Materials

Jacob Hauck

June 3, 2026

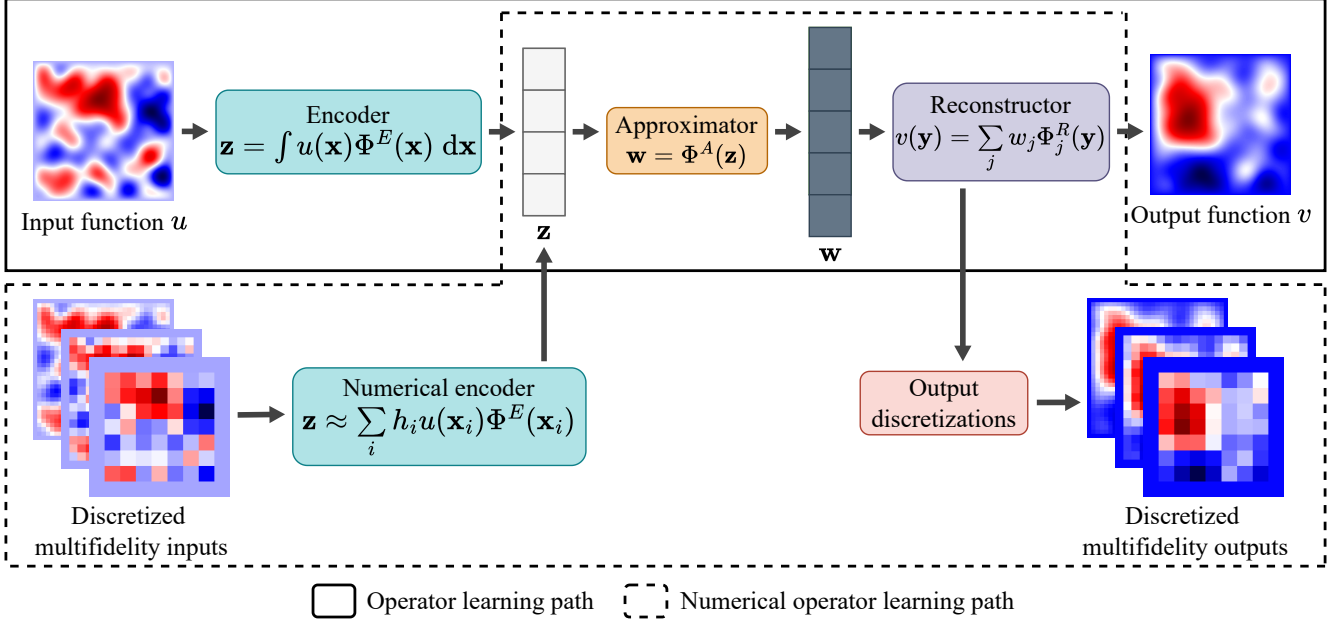


Figure 1: The architecture of our operator learning method, and how operator learning enables discretization independence through numerical operator learning. Note the three-module structure: (linear) encoder projects the input function onto a finite-dimensional space parameterized by neural network basis functions (Φ^E), approximator approximates the operator in the latent space, and (linear) reconstructor builds the output function by taking a linear combination of neural network basis functions (using a different neural network Φ^R) using the latent output variables from the approximator as weights.

Approximator	Error on training data	Error on testing data
Base (3 layers, 128 neurons/layer, ReLU)	15.2	18.1
Wide (3 layers, 384 neurons/layer, ReLU)	12.4	20.7
Deep (8 layers, 128 neurons/layer, SELU)	11.9	16.9

Table 1: Error (measured in % relative L^2) for different approximator network settings. Using a wide approximator leads to overfitting (note gap between train and test error in the second row), but using a deep approximator with self-normalizing activation leads to a slight performance boost (compare error in first and third rows).

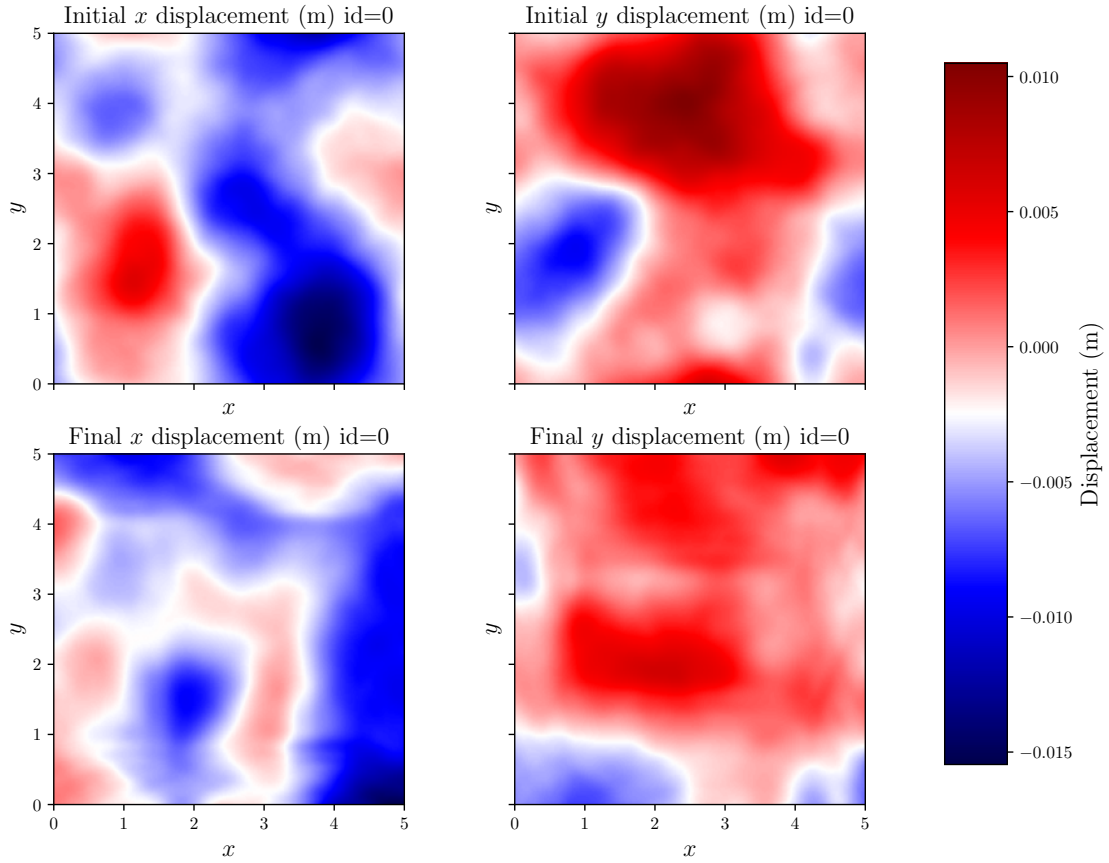


Figure 2: Example input–output pair from one of the wave propagation benchmark datasets, prepared using PDMATLAB2D. The input function is the top row, which shows the initial x and y displacement fields. The output function is the bottom row, which shows the final x and y displacement fields after a time $T = 1.8$ seconds has elapsed in the simulation. The “id=0” refers to this sample’s ID in the dataset.

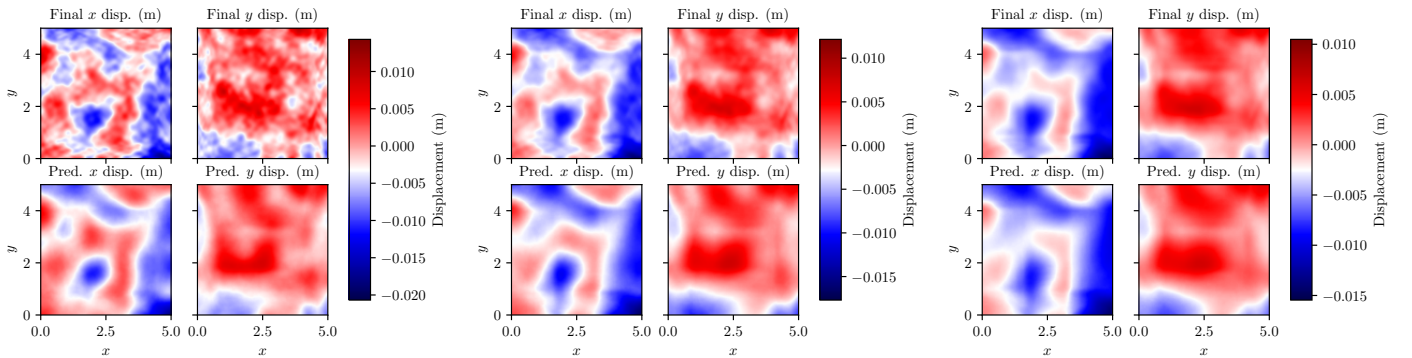
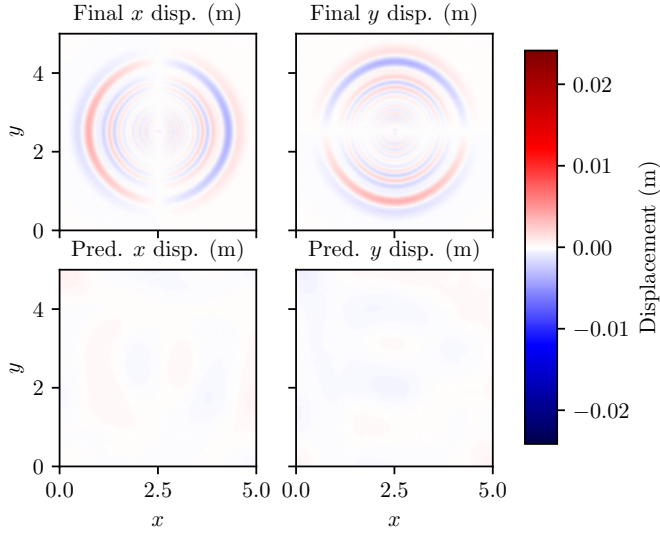
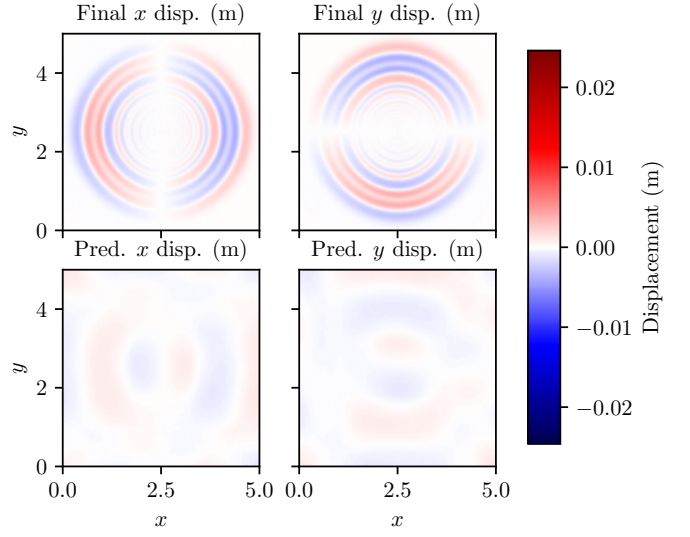


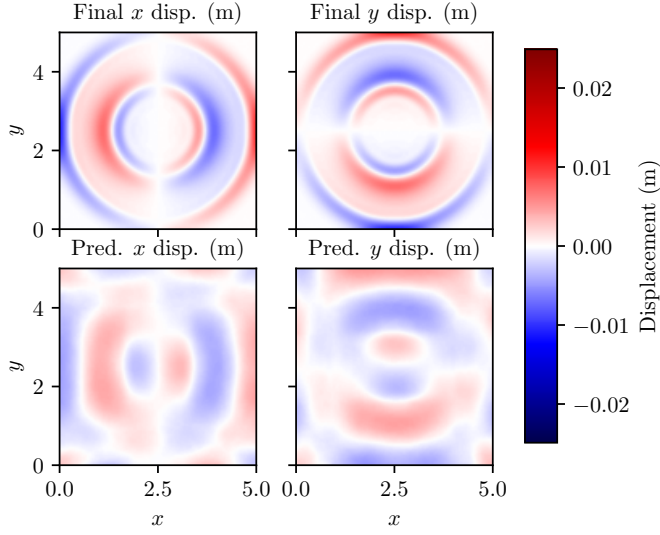
Figure 3: Example final displacement fields (top row, x and y components alternating) and corresponding model predictions (bottom row) for different input Gaussian process covariance eigenvalue decay rates. Decay rate increases from left to right. Note the corresponding decrease in strong high-frequency components from left to right. Also note how the model prediction only capture low-frequency (smooth) components, making its predictions more accurate when the covariance eigenvalues decay quickly.



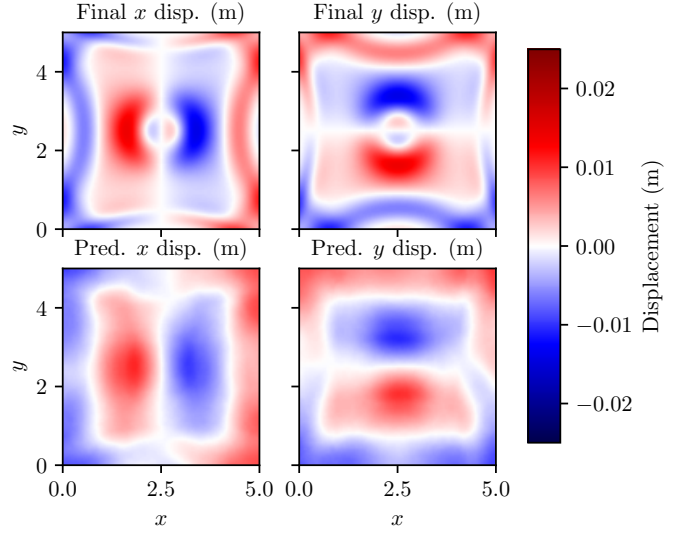
(a) Initial displacement scale = 0.03



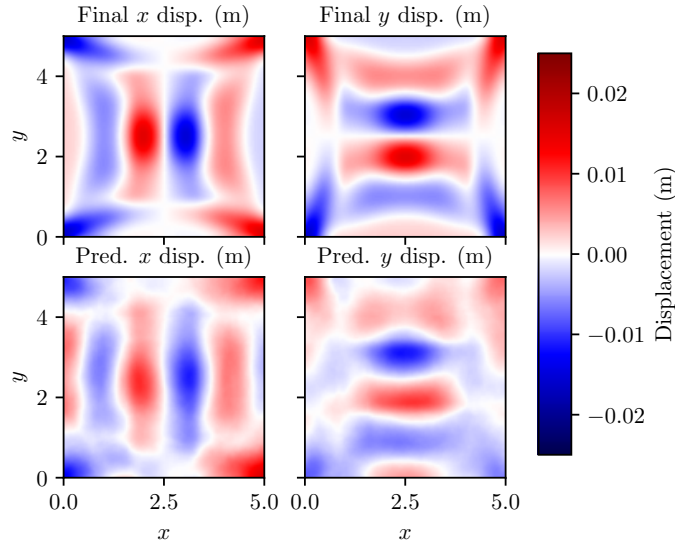
(b) Initial displacement scale = 0.05



(c) Initial displacement scale = 0.10



(d) Initial displacement scale = 0.20



(e) Initial displacement scale = 0.30

Figure 4: Model predictions for out-of-distribution test initial displacements. The top row in each block shows simulation final displacement field x and y components, and the bottom rows show the corresponding model predictions. Input initial displacement fields are Gaussian bumps of varying sizes, specified in the captions of each subfigure. The model makes more accurate predictions for larger bumps (which have less high-frequency information and result in less dispersive effects).

T	Error on training data	Error on testing data
1.8	15.4	18.2
1.4	16.5	21.0
1.0	16.0	19.3
0.6	14.1	17.1
0.2	14.4	19.2

Table 2: Independence of model error (measured in % relative L^2) on simulation duration T . When $T = 0.2$, the operator is nearly the identity, suggesting that the reconstructor module in our model is insufficient. Addressing this limitation will be the focus of the remaining work on the wave propagation problem.

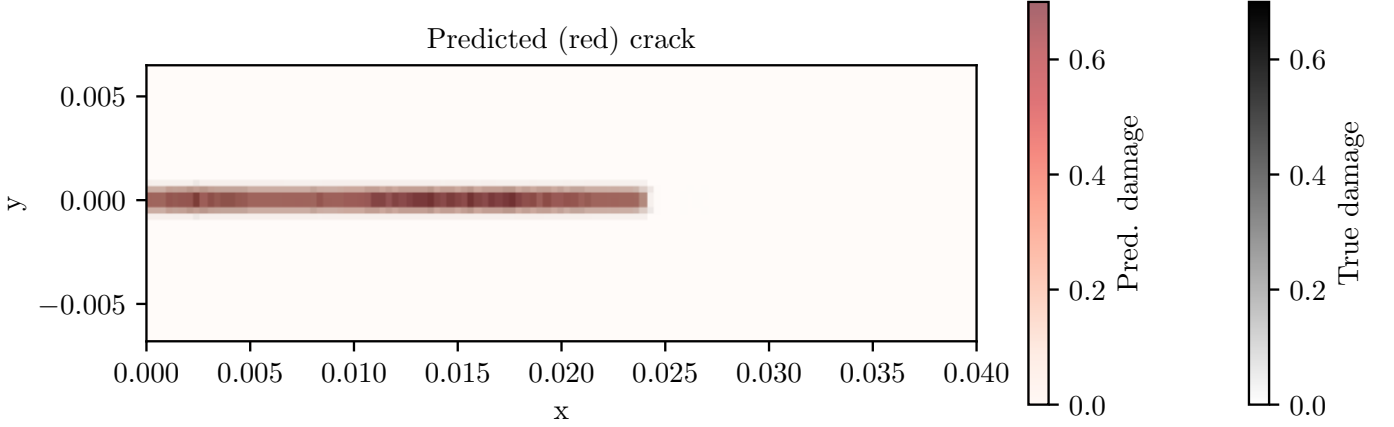


Figure 5: An example crack final damage field (black) and model prediction superimposed (red) in the case of symmetric traction on the top and bottom of the plate. This image is cropped to show only the part of the crack that grew. Note that the damage field refers to the fraction of broken bonds (between 0 and 1) per particle. The model prediction agrees closely with the true crack path.

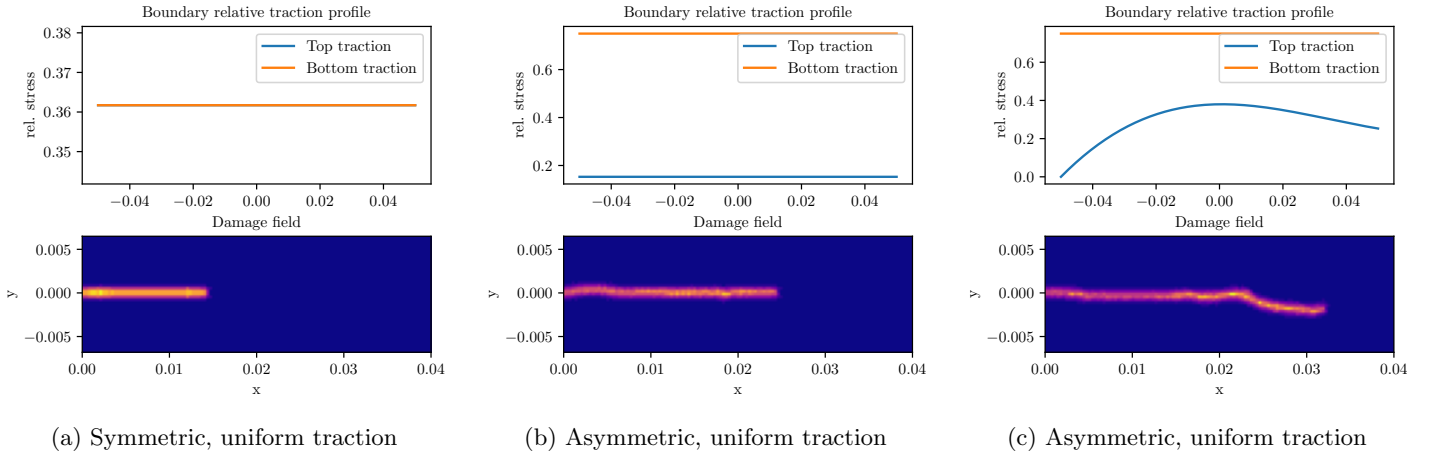


Figure 6: Samples from generated crack propagation datasets. The input function is the boundary relative traction profile (top row, units are relative to a fixed, maximum traction), and the output function is the final damage field (bottom row, cropped to the region around where the crack grew). Note that dark blue means no damage, and orange/white means high damage. The goal for operator learning on these datasets is to predict the final crack path.

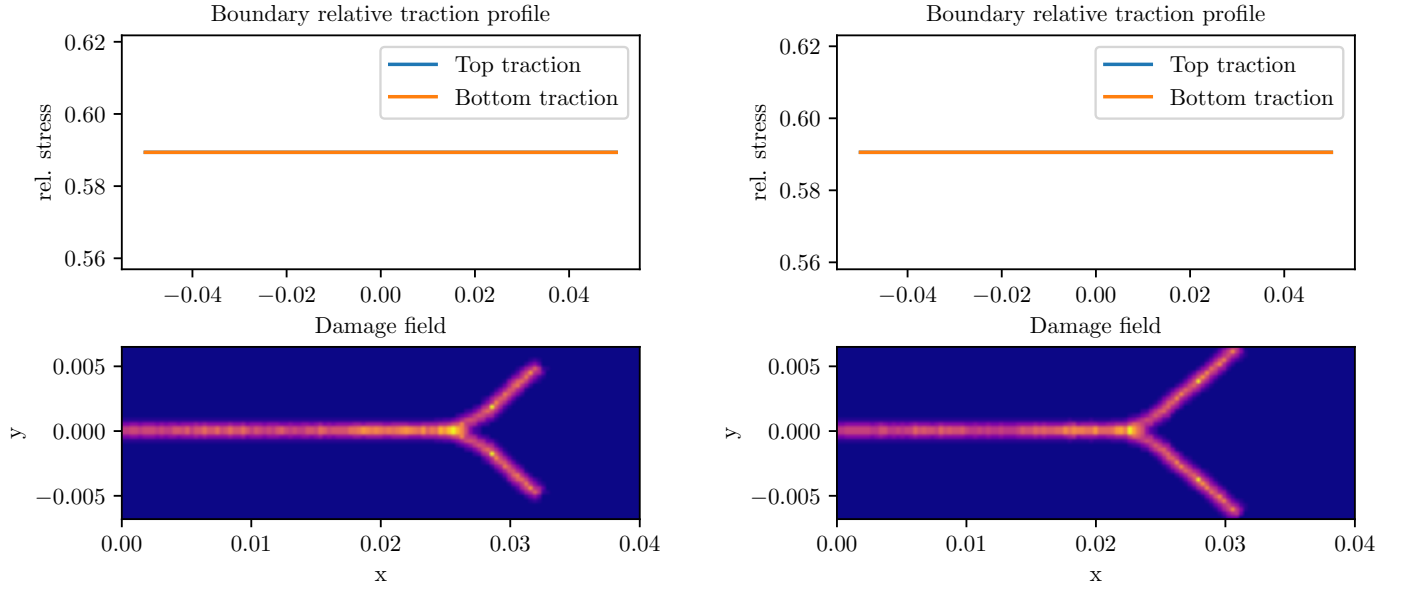


Figure 7: Comparison of simulated crack paths with symmetric, uniform relative traction that differs in strength by less than 1%. The simulation produces very different results despite the nearly identical parameters. This makes prediction with operator learning nearly impossible, so finding a stable configuration is a top priority for the remaining time in my award.

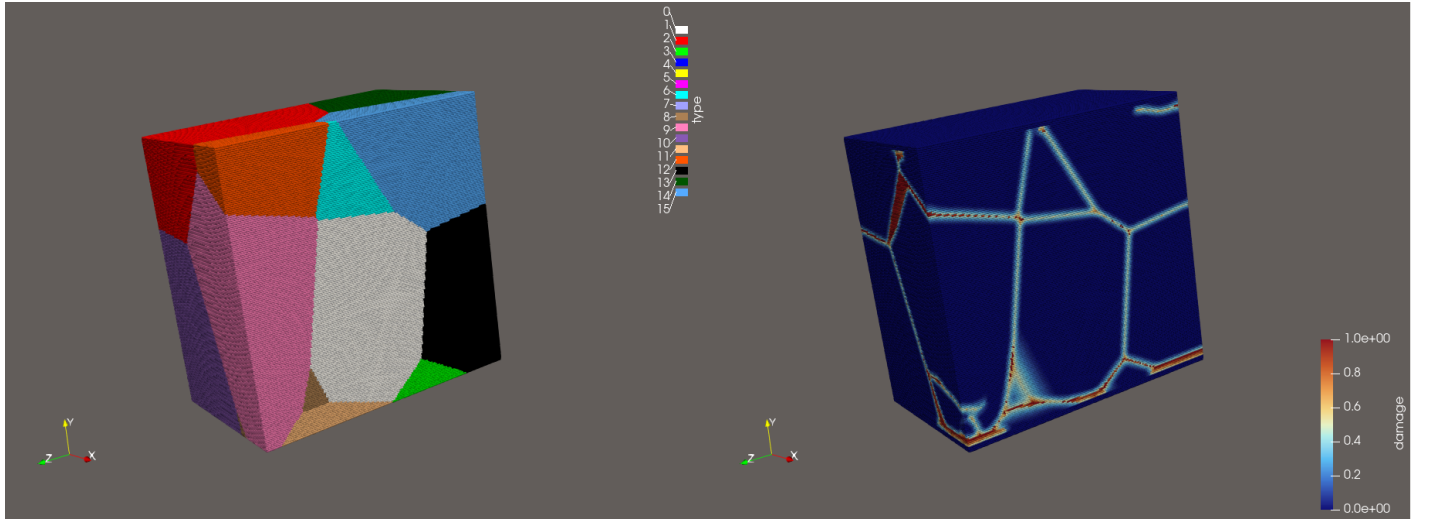


Figure 8: Grain structure (left) and final damage field (right) in a small, cube-shaped polycrystal with 16 grains. Simulation was performed with CabanaPD. This view shows a cross-section of the cube perpendicular to the x axis.